WARSZTAT PRAKTYCZNY

Claude lub OpenAI prosto z T-SQL:
asystent opinii gości hotelowych

Realny scenariusz dla fikcyjnej sieci „Hotele Bursztyn”. Krok po kroku budujemy w bazie jeden spójny mechanizm: od pierwszego wywołania API przez `sp_invoke_external_rest_endpoint`, przez analizę opinii do ustrukturyzowanego JSON, przetwarzanie wsadowe z kontrolą kosztów, aż po szkice odpowiedzi chronione przed wyciekiem danych osobowych i prompt injection.

Parametr	Wartość
Szkolenie	Wykorzystanie AI w pracy z SQL Server
Grupa docelowa	Zaawansowani użytkownicy SQL / T-SQL; doświadczenie z API REST niewymagane
Środowisko	SQL Server 2025 (17.x) Developer Edition, SSMS, dostęp do internetu z serwera (HTTPS)
Dostawca modelu	Do wyboru przez kursanta: Anthropic (Claude) lub OpenAI – logika warsztatu jest wspólna
Baza	`AsystentOpinii` – tworzona od zera skryptem z Modułu 1 (opinie fikcyjne)
Struktura	4 moduły; każdy rozbudowuje ten sam mechanizm i kończy się weryfikacją OK / BŁĄD

Spis treści

Spis treści 2
0. Informacje 3
Cele dydaktyczne 3
Jak rośnie mechanizm w kolejnych modułach 3
Agenda 3
Scenariusz biznesowy 3
Wybór dostawcy: Claude czy OpenAI? 4
Model danych warsztatu 4
Moduł 1. Fundament: pierwsze wywołanie i procedura bazowa 5
Ćwiczenie 1.1 – środowisko i dane 5
Ćwiczenie 1.2 – poświadczenie z kluczem API 5
Ćwiczenie 1.3 – pierwsze „surowe” wywołanie 5
Ćwiczenie 1.4 – procedura bazowa usp_LLM_Zapytaj 5
Weryfikacja Modułu 1 6
Moduł 2. Ustrukturyzowana analiza opinii 7
Ćwiczenie 2.1 – prompt jako obiekt w bazie 7
Ćwiczenie 2.2 – procedura analizy jednej opinii 7
Ćwiczenie 2.3 – pierwsze pięć opinii 7
Weryfikacja Modułu 2 8
Moduł 3. Przetwarzanie wsadowe, koszty i raport 9
Ćwiczenie 3.1 – procedura wsadowa 9
Ćwiczenie 3.2 – analiza archiwum 9
Ćwiczenie 3.3 – raport dla zarządu 9
Weryfikacja Modułu 3 10
Moduł 4. RODO, szkice odpowiedzi i cały proces 11
Ćwiczenie 4.1 – audyt: co wysłaliśmy dostawcy? 11
Ćwiczenie 4.2 – maskowanie w jednym miejscu 11
Ćwiczenie 4.3 – szkice odpowiedzi z drugą linią obrony 11
Ćwiczenie 4.4 – cały proces i nowe opinie 11
Weryfikacja Modułu 4 12
Podsumowanie 13
Co zbudowaliśmy 13
Pytania do dyskusji 13
Zadania dodatkowe dla chętnych 13
Pytania kontrolne (z odpowiedziami) 13
Co przygotować przed zajęciami 14
Załącznik. Sprzątanie po warsztacie 14
Źródła 14

0. Informacje

Cele dydaktyczne

Po warsztacie uczestnik:

- wywołuje API modelu językowego z T-SQL przez `sp_invoke_external_rest_endpoint`, bezpiecznie przechowując klucz w poświadczeniu bazy,
- buduje żądania funkcją `JSON_OBJECT` i odczytuje odpowiedzi przez `OPENJSON`, odporne na różnice między dostawcami,
- projektuje prompt zwracający ustrukturyzowany JSON i waliduje wynik w bazie zamiast ufać modelowi,
- przetwarza dane wsadowo z idempotentnością (cache po hashu treści i wersji promptu), budżetem kosztów i obsługą błędów,
- wykrywa i usuwa wyciek danych osobowych do dostawcy, broni się przed prompt injection i utrzymuje człowieka w pętli decyzyjnej.

Jak rośnie mechanizm w kolejnych modułach

Moduł	Co dobudowujemy	Na czym bazuje
1	`usp_LLM_Zapytaj` – jedyne miejsce rozmowy z modelem: konfiguracja dostawcy, poświadczenie, dziennik, koszty, pusty „hak” prywatności	–
2	`usp_AnalizujOpinie` – wersjonowany prompt, JSON, walidacja, ponowienie, zapis do tabeli	`usp_LLM_Zapytaj`
3	`usp_AnalizujWszystkie` – kolejka, cache, budżet, błędy; raporty dla zarządu	`usp_AnalizujOpinie`
4	Maskowanie danych w „haku”, `usp_PrzygotujOdpowiedz`, reguły biznesowe, proces `usp_ObsluzNoweOpinie`	wszystkie powyższe

Agenda

Blok	Treść
Wprowadzenie	Scenariusz, architektura, wybór dostawcy
Moduł 1	Fundament: pierwsze wywołanie i procedura bazowa
Moduł 2	Ustrukturyzowana analiza opinii (JSON)
Moduł 3	Przetwarzanie wsadowe, koszty, raport
Moduł 4	RODO, szkice odpowiedzi, prompt injection, cały proces
Zamknięcie	Dyskusja, pytania kontrolne

Scenariusz biznesowy

Sieć „Hotele Bursztyn” (Gdańsk, Kraków, Zakopane) zbiera opinie gości z kilku portali rezerwacyjnych. Dział jakości czyta je ręcznie, z opóźnieniem kilku dni – ostatnio pilna sprawa (pluskwy w pokoju) czekała na reakcję tydzień. Dyrektor operacyjna chce, aby każda opinia była od razu sklasyfikowana (sentyment, kategoria, pilność), aby powstawał raport problemów każdego hotelu, a na opinie negatywne przygotowywały się szkice odpowiedzi do zatwierdzenia przez kierownika.

Dane są już w SQL Server, więc logikę budujemy tam, gdzie są dane: procedury T-SQL wywołują API modelu językowego bezpośrednio przez `sp_invoke_external_rest_endpoint`. Nie potrzeba dodatkowej aplikacji ani serwera pośredniczącego.

Wybór dostawcy: Claude czy OpenAI?

Proszę wybrać jednego dostawcę w ćwiczeniu 1.2. Różnice są zamknięte w jednej procedurze bazowej – wszystkie kolejne moduły działają tak samo.

Element	Anthropic (Claude)	OpenAI
Endpoint	`https://api.anthropic.com/v1/messages`	`https://api.openai.com/v1/responses`
Uwierzytelnienie	nagłówek `x-api-key`	nagłówek `Authorization: Bearer …`
Wersja API	nagłówek `anthropic-version: 2026-10-21`	–
Model w warsztacie	`claude-haiku-5-5`	`gpt-6-luna`
Instrukcja systemowa	pole `system`	pole `instructions`
Treść pytania	`messages: [{role, content}]`	`input`
Tekst odpowiedzi	`content[]` → bloki `type = text`	`output[]` → `message` → `content[]` → `output_text`
Tokeny	`usage.input_tokens`, `usage.output_tokens`	`usage.input_tokens`, `usage.output_tokens`

Ważne: nazwy modeli, ceny i wersje API zmieniają się często. Wartości w tabeli `dbo.KonfiguracjaLLM` (model, ceny za 1 mln tokenów) sprawdź tuż przed zajęciami w dokumentacji dostawcy i w razie potrzeby popraw jednym UPDATE. W wersji `2026-10-21` API Anthropic usunięto parametr `temperature` – dlatego nie używamy go w warsztacie, a weryfikacje tolerują naturalną zmienność odpowiedzi modelu (progi procentowe zamiast dokładnych wartości).

Model danych warsztatu

Tabela / obiekt	Zawartość	Moduł
`dbo.KonfiguracjaLLM`	Dostawcy, endpointy, modele, ceny; dokładnie jeden aktywny	1
`dbo.LogLLM`	Każde wywołanie: czas, HTTP, tokeny, koszt, wysłane zapytanie, odpowiedź, błąd	1
`dbo.Hotel`, `dbo.Opinia`	3 hotele, 40 opinii (w tym 3 z danymi osobowymi, 2 po angielsku, 1 jednowyrazowa)	1
`dbo.KluczOpinii`	Oczekiwany sentyment i kategoria dla opinii jednoznacznych – tylko do weryfikacji	1
`dbo.fn_FiltrPrywatnosci`	Funkcja-„hak” wywoływana przed każdym wysłaniem tekstu; w Module 1 pusta	1 → 4
`dbo.PromptSzablon`	Wersjonowane prompty systemowe	2
`dbo.AnalizaOpinii`	Sentyment, kategorie, pilność, język, streszczenie, hash treści, wersja promptu	2
`dbo.BladPrzetwarzania`	Błędy przetwarzania wsadowego	3
`dbo.OdpowiedzNaOpinie`	Szkice odpowiedzi ze statusem i powodem odrzucenia	4

Moduł 1. Fundament: pierwsze wywołanie i procedura bazowa

Cel modułu
Przygotować bazę i dane, bezpiecznie zapisać klucz API, wykonać pierwsze wywołanie z T-SQL i zbudować procedurę bazową z dziennikiem i liczeniem kosztów.

Ćwiczenie 1.1 – środowisko i dane

Skrypt wymaga uprawnień sysadmin (włącza na serwerze opcję `external rest endpoint enabled`). Usuwa bazę `AsystentOpinii`, jeśli istnieje – warsztat można zacząć od nowa w każdej chwili.

Skrypt: m1_setup.sql

Na co zwrócić uwagę: konfiguracja dostawcy jest w tabeli, a nie w kodzie procedur – zmiana modelu lub dostawcy nie wymaga zmian w logice. Filtrowany indeks unikalny gwarantuje, że aktywny jest dokładnie jeden dostawca. Funkcja `fn_FiltrPrywatnosci` na razie zwraca tekst bez zmian – to celowy „hak", który wypełnimy w Module 4.

Ćwiczenie 1.2 – poświadczenie z kluczem API

Klucz API trafia do `DATABASE SCOPED CREDENTIAL`, zaszyfrowanego kluczem głównym bazy. Nazwa poświadczenia musi być adresem URL (schemat i host zgodne z wywoływanym adresem) – dzięki temu SQL Server dołącza sekret tylko do żądań kierowanych do tego hosta. Wklej swój klucz i uruchom wariant swojego dostawcy.

Skrypt: m1_credential.sql

Zwróć uwagę grupy, że `sys.database_scoped_credentials` nie pokazuje sekretu. **Klucza nie zapisujemy w repozytorium ani w tabelach** – po zajęciach unieważnij klucz w konsoli dostawcy.

Ćwiczenie 1.3 – pierwsze „surowe" wywołanie

Zanim schowamy wszystko w procedurze, zobaczmy, co naprawdę wraca z API. Przykład jest dla Claude; wariant OpenAI jest w komentarzu na końcu skryptu.

Skrypt: m1_raw.sql

Oczekiwany wynik: `KodPowrotu` = 0, `KodHTTP` = 200. Pełna odpowiedź ma strukturę `{"response": {"status": …, "headers": …}, "result": {…}}` – w `result` jest dokładnie to, co zwróciło API. W tablicy `content` może pojawić się więcej niż jeden blok (np. blok rozumowania modelu przed blokiem `text`), dlatego tekst wybieramy po typie, a nie jako pierwszy element.

Ćwiczenie 1.4 – procedura bazowa usp_LLM_Zapytaj

To serce warsztatu. Każdy kolejny moduł korzysta tylko z tej procedury, więc tu raz rozwiązujemy: wybór dostawcy, budowę żądania, ponowienia, odczyt tekstu, tokeny, koszt, dziennik i obsługę błędów.

Skrypt: m1_core.sql

Oczekiwany wynik: odpowiedź „Warszawa" oraz wiersz w `dbo.LogLLM` z kodem 200, liczbą tokenów (kilkadziesiąt wejściowych, kilka–kilkadziesiąt wyjściowych) i kosztem rzędu ułamka centa.

> **Krytycznym okiem: gdy kod pisze asystent AI**
>
> Przykładowy prompt: „Napisz procedurę T-SQL, która wywoła API Claude/OpenAI i zwróci odpowiedź".
>
> - Klucz API wpisany na sztywno w `@headers` lub zapisany w tabeli konfiguracyjnej – wycieka do historii zapytań, kopii zapasowych i dziennika. Właściwe miejsce: poświadczenie bazy.

- Żądanie sklejane z tekstu (`'{"content":"' + @Prompt + '"}'`) – pierwszy cudzysłów lub znak nowej linii w opinii psuje JSON. Używaj `JSON_OBJECT`.
- Odczyt `JSON_VALUE(@r, '$.result.content[0].text')` – zawodzi, gdy pierwszy blok nie jest tekstem, i obcina wynik do 4000 znaków. Używaj `OPENJSON … WITH (… nvarchar(max))`.
- Parametry, których API już nie przyjmuje (np. `temperature` w najnowszej wersji API Anthropic) albo nieistniejące nazwy modeli – model AI zna stan z daty swojej wiedzy, nie z dziś.
- Brak dziennika: bez niego nie policzysz kosztów, nie przeprowadzisz audytu RODO (Moduł 4) i nie zdiagnozujesz błędów.

Weryfikacja Modułu 1

Skrypt: m1_verify.sql

Weryfikacja – czy wszystko działa?

Wszystkie 10 testów powinno mieć status **OK**. Test 10 wykonuje jedno prawdziwe wywołanie API.

- Test 4 = BŁĄD → nazwa poświadczenia różni się od `NazwaPoswiadczenia` w konfiguracji (np. końcowy ukośnik).
- Test 10 z komunikatem HTTP 401 → błędny klucz; HTTP 404 lub 400 z „model” → popraw nazwę modelu w `dbo.KonfiguracjaLLM`.
- Błąd połączenia (wyjątek zamiast kodu HTTP) → serwer nie ma wyjścia do internetu lub blokuje go zapora / proxy firmowe.

Moduł 2. Ustrukturyzowana analiza opinii

Cel modułu
Zamienić wolny tekst opinii w wiersz tabeli: wersjonowany prompt, odpowiedź w JSON, walidacja w T-SQL, ponowienie przy błędzie i zapis do dbo.AnalizaOpinii.

Ćwiczenie 2.1 – prompt jako obiekt w bazie

Skrypt: m2_prompt.sql

Omów z grupą budowę promptu: rola, wyraźne oddzielenie danych znacznikami `<opinia>`, zakaz wykonywania poleceń z treści, dokładny format JSON, zamknięte słowniki wartości i definicja pilności z przykładami. Prompt ma wersję – w Module 3 zobaczymy, dlaczego to ważne (zmiana promptu = ponowna analiza).

Ćwiczenie 2.2 – procedura analizy jednej opinii

Plik / skrypt: m2_proc.sql

Kluczowa zasada: odpowiedź modelu traktujemy jak dane z niezaufanego źródła. `ISJSON(…, OBJECT)`, słowniki wartości i zakres pilności sprawdzamy w T-SQL. Gdy walidacja nie przejdzie, wysyłamy drugą próbę z informacją zwrotną; gdy i ona zawiedzie – zgłaszamy błąd, a nie zapisujemy śmieci.

Ćwiczenie 2.3 – pierwsze pięć opinii

Skrypt: m2_run.sql

Oczekiwany wynik (sformułowania streszczeń będą się różnić):

Opinia	Sentyment	Kategoria główna	Pilność	Język
1 – widok na morze, wygodne łóżko	pozytywny	pokoj (lub lokalizacja)	1	pl
2 – brudny pokój	negatywny	czystosc	3–4	pl
3 – świetne śniadania	pozytywny	jedzenie	1	pl
4 – Great location…	pozytywny	lokalizacja	1	en
5 – miła obsługa, ale wesele	mieszany	halas	2–3	pl

Dyskusja: opinia 1 równie dobrze pasuje do kategorii „pokój” i „lokalizacja”. Model nie jest „nieomylną etykietą” – dlatego weryfikacja w Module 3 mierzy trafność na opiniach jednoznacznych i akceptuje próg, a nie 100%.

Krytycznym okiem: gdy kod pisze asystent AI

Przykładowy prompt: „Napisz prompt, który zwróci sentyment i kategorię opinii w JSON”.

- Brak zamkniętego słownika wartości: model zwraca raz „negative”, raz „negatywna”, raz „zły” – raport w Module 3 się rozsypie.
- Brak walidacji po stronie bazy i zapis `JSON_VALUE(…)` prosto do tabeli – pierwszy blok ```json lub komentarz modelu kończy się wartościami NULL.
- Treść opinii doklejona do instrukcji bez separatorów – opinia z poleceniem „zignoruj instrukcje” może przejąć kontrolę (Moduł 4).
- Pilność bez definicji: „pilność 1–5” bez przykładów to subiektywna ocena modelu, różna przy każdym wywołaniu.

Weryfikacja Modułu 2

Skrypt: m2_verify.sql

Weryfikacja – czy wszystko działa?

Wszystkie 8 testów powinno mieć status **OK**.

- Test 2 < 5 → przynajmniej jedno wywołanie zakończyło się błędem; sprawdź ostatnie wpisy w `dbo.LogLLM` (kolumny `Blad`, `Odpowiedz`).
- Test 4 = BŁĄD → przejrzyj sentymenty; jeśli model systematycznie myli „mieszany” z „negatywny”, doprecyzuj definicje w promptcie i zapisz jako wersję 2.
- Komunikat „Odpowiedź ucięta” → zwiększ `@MaxTokens` w wywołaniu (model mógł zużyć tokeny na rozumowanie).

Moduł 3. Przetwarzanie wsadowe, koszty i raport

Cel modułu
Przeanalizować całe archiwum bez dublowania wywołań, z budżetem kosztów i odpornością na błędy, a następnie zbudować raport dla zarządu.

Ćwiczenie 3.1 – procedura wsadowa

- **Idempotentność:** przetwarzamy tylko opinie bez aktualnej analizy – nowe, zmienione (inny hash SHA-256 treści) lub analizowane starszą wersją promptu. Ponowne uruchomienie nie kosztuje nic.
- **Budżet:** przed każdym wywołaniem sumujemy koszt z dziennika; po przekroczeniu limitu proces zatrzymuje się w kontrolowany sposób.
- **Odporność:** błąd jednej opinii trafia do `dbo.BladPrzetwarzania`, a reszta jest przetwarzana dalej.

Skrypt: m3_proc.sql

Ćwiczenie 3.2 – analiza archiwum

Skrypt: m3_run.sql

Oczekiwany wynik: pierwsze uruchomienie – ok. 35 przeanalizowanych opinii (pozostałe 5 zrobiliśmy w Module 2), 0 błędów, koszt rzędu tysięcznych lub setnych części dolara (zależnie od modelu i cennika), czas 1–3 minut. Drugie uruchomienie – 0 przeanalizowanych, 0 wywołań API, koszt 0.

Pytanie do grupy: co się stanie, gdy poprawimy prompt i zapiszemy go jako wersję 2? (Wszystkie 40 opinii trafi ponownie do kolejki – i dobrze, bo stare wyniki powstały według innych zasad. Dzięki budżetowi wiemy z góry, ile to maksymalnie kosztuje.)

Ćwiczenie 3.3 – raport dla zarządu

Skrypt: m3_report.sql

Czego się spodziewać: w zapytaniu b) na liście pilnych są m.in. pluskwy (6), kradzież laptopa (14), brązowa woda i wysypka (26). Zapytanie c) szuka rozbieżności między gwiazdkami a treścią; w naszych danych zwykle zwraca niewiele wierszy albo żadnego, ale w realnych danych to cenna informacja dla działu jakości (np. ocena 5 z treścią pełną skarg). Zapytanie d) pokazuje koszt każdego etapu – argument w rozmowie o budżecie projektu.

> **Krytycznym okiem: gdy kod pisze asystent AI**
>
> Przykładowy prompt: „Napisz pętlę, która przeanalizuje wszystkie opinie przez API”.
>
> - Pętla bez warunku „czy już przeanalizowano” – każde uruchomienie płaci za wszystko od nowa.
> - Jeden TRY/CATCH wokół całej pętli – pierwszy błąd HTTP zatrzymuje proces, reszta opinii zostaje bez analizy.
> - Wysyłanie wszystkich opinii w jednym ogromnym zapytaniu „dla oszczędności” – trudna walidacja, ryzyko pomylenia wyników między opiniami i ucięcia odpowiedzi.
> - Brak limitu kosztów i liczby wywołań: błąd w warunku pętli może wygenerować tysiące płatnych wywołań.

Weryfikacja Modułu 3

Skrypt: m3_verify.sql

Weryfikacja – czy wszystko działa?

Wszystkie 7 testów powinno mieć status **OK**.

- Test 1 < 40 → sprawdź `dbo.BladPrzetwarzania` i uruchom `usp_AnalizujWszystkie` ponownie – przetworzy tylko brakujące.
- Test 2 lub 3 = BŁĄD → odchylenia od klucza są możliwe przy granicznych opiniach; przejrzyj różnice zapytaniem z JOIN do `dbo.KluczOpinii` i omów je z grupą.
- Test 7 = BŁĄD → warunek w kursorze nie wyklucza przeanalizowanych opinii (porównanie hasha lub wersji promptu).

Moduł 4. RODO, szkice odpowiedzi i cały proces

Cel modułu
Wykryć i usunąć wyciek danych osobowych, generować szkice odpowiedzi odporne na prompt injection, zatwierdzane przez człowieka, i spiąć wszystko w jeden proces.

Ćwiczenie 4.1 – audyt: co wysłaliśmy dostawcy?

Dziennik z Modułu 1 zapisuje dokładnie to, co wyszło do API. Sprawdźmy, czy nie było tam danych osobowych. Funkcje `REGEXP_LIKE` i `REGEXP_SUBSTR` są dostępne od SQL Server 2025.

Skrypt: m4_audit.sql

Oczekiwany wynik: wpisy z Modułu 3 dla opinii 20, 32, 70 – adresy e-mail i numery telefonów gości trafiły do zewnętrznego dostawcy. To realny incydent: w praktyce trzeba go ocenić pod kątem RODO (podstawa prawna, umowa powierzenia z dostawcą, polityka przechowywania danych po jego stronie).

Ćwiczenie 4.2 – maskowanie w jednym miejscu

Ponieważ każde wywołanie przechodzi przez `usp_LLM_Zapytaj`, wystarczy uzupełnić funkcję-„hak” z Modułu 1. Od tej chwili chronione są wszystkie obecne i przyszłe procedury.

Skrypt: m4_mask.sql

Oczekiwany wynik testu: `Kontakt: [EMAIL], tel. [TELEFON] lub [TELEFON]. Pokój 214, 650 zł.` – numer pokoju i cena zostają. **Uwaga:** wyrażenia regularne nie wykryją imion, nazwisk ani adresów zapisanych słowami. To minimum, nie pełna anonimizacja.

Ćwiczenie 4.3 – szkice odpowiedzi z drugą linią obrony

Skrypt: m4_reply.sql

Model dostaje wynik analizy z Modułu 2, więc pisze odpowiedź dopasowaną do pilności. Po wygenerowaniu T-SQL sprawdza reguły biznesowe niezależnie od modelu: podejrzane kody, obietnice rekompensat, brak podpisu, długość. Szkic nigdy nie jest publikowany automatycznie – trafia do statusu „Do akceptacji” albo „Do poprawy”.

Ćwiczenie 4.4 – cały proces i nowe opinie

Procedura `usp_ObsluzNoweOpinie` łączy wszystko, co zbudowaliśmy: analizę przyrostową (Moduł 3, a pod spodem 2 i 1) oraz szkice odpowiedzi. Wśród trzech nowych opinii jest próba prompt injection (43) i opinia z numerem telefonu (41).

Skrypt: m4_pipeline.sql

Oczekiwany wynik: analiza 3 nowych opinii, a następnie szkice dla wszystkich opinii negatywnych, mieszanych i pilnych (ok. 20 wywołań). Opinia 41 ma pilność ≥ 4, a w dzienniku zamiast numeru widać `[TELEFON]`. Szkic dla opinii 43 odnosi się do klimatyzacji i nie przyznaje kodu GRATIS-500 – a jeśli model jednak by go powtórzył, reguła w T-SQL ustawi status „Do poprawy”.

> **Krytycznym okiem: gdy kod pisze asystent AI**
>
> Przykładowy prompt: „Napisz procedurę, która automatycznie odpowiada na negatywne opinie przez API”.
>
> - Automatyczna publikacja bez akceptacji człowieka – jedna halucynacja (np. obiecany zwrot pieniędzy) staje się publicznym zobowiązaniem firmy.

- Ochrona przed prompt injection tylko w promptcie systemowym. Model zwykle się obroni, ale nie zawsze – potrzebna jest druga, deterministyczna linia obrony w kodzie.
- Maskowanie danych osobowych w każdej procedurze osobno (i zapomnienie o jednej) zamiast w jednym, centralnym miejscu.
- Przekonanie, że regex „anonimizuje dane" – maskuje tylko to, co da się opisać wzorcem.

Weryfikacja Modułu 4

Skrypt: m4_verify.sql

Weryfikacja – czy wszystko działa?

Wszystkie 8 testów powinno mieć status **OK**, a podsumowanie: „Moduł 4 zaliczony – warsztat ukończony!".

- Test 2 = BŁĄD → nie wykonano `UPDATE` czyszczącego dziennik w ćw. 4.2 albo funkcja maskująca nie została zmieniona.
- Test 5 = BŁĄD → część szkiców zakończyła się błędem; sprawdź `dbo.BladPrzetwarzania` i uruchom `usp_ObsluzNoweOpinie` ponownie.
- Test 6 = BŁĄD → model wykonał polecenie z opinii, a reguła w T-SQL go nie wychwyciła. Świetny materiał do dyskusji: wzmocnij regułę i prompt (wersja 2).
- Test 8 = BŁĄD → sprawdź `PowodStatusu`; reguła słów kluczowych może być zbyt ostra (np. odrzuca uprzejmą odmowę rabatu). Dostrój ją świadomie.

Podsumowanie

Co zbudowaliśmy

- Jedną procedurę bazową, przez którą przechodzi każde wywołanie modelu: konfiguracja, sekrety, dziennik, koszty, ochrona prywatności.
- Analizę opinii z walidacją wyniku i wersjonowanym promptem.
- Idempotentne przetwarzanie wsadowe z budżetem i raportem dla zarządu.
- Proces end-to-end ze szkicami odpowiedzi, dwiema liniami obrony i człowiekiem w pętli.

Pytania do dyskusji

- Jakie dane z Waszych baz wolno wysłać do zewnętrznego modelu, a jakie muszą zostać w organizacji? Kto o tym decyduje?
- Kiedy wywołanie API z T-SQL ma sens, a kiedy lepsza jest osobna usługa (kolejki, długie przetwarzanie, równoległość)?
- Jak mierzyć jakość modelu w czasie? (Wskazówka: stały zestaw opinii z kluczem uruchamiany po każdej zmianie promptu lub modelu – tak jak nasze weryfikacje.)

Zadania dodatkowe dla chętnych

- Przełącz dostawcę jednym `UPDATE` w `dbo.KonfiguracjaLLM`, przeanalizuj opinie ponownie (wersja promptu 2) i porównaj wyniki obu modeli.
- Dodaj tabelę zatwierdzeń: kierownik zmienia status na „Opublikowana”, a trigger zapisuje, kto i kiedy to zrobił.
- Zaplanuj `usp_ObsluzNoweOpinie` jako zadanie SQL Server Agent co godzinę z dziennym budżetem.
- Połącz warsztat z poprzednim: generuj embeddingi opinii i wyszukuj podobne skargi z innych hoteli.

Pytania kontrolne (z odpowiedziami)

Pytanie	Odpowiedź
Gdzie przechowujemy klucz API i dlaczego?	W `DATABASE SCOPED CREDENTIAL` zaszyfrowanym kluczem głównym bazy – nie trafia do kodu, tabel, dziennika ani historii zapytań.
Dlaczego żądanie budujemy przez `JSON_OBJECT`?	Funkcja poprawnie escapuje cudzysłowy i znaki specjalne z treści opinii; sklejanie tekstu psuje JSON.
Jak zapewniamy, że ponowne uruchomienie nic nie kosztuje?	Analizujemy tylko opinie bez wyniku dla bieżącej wersji promptu i bieżącego hasha treści.
Dlaczego walidujemy JSON w T-SQL, skoro prompt opisuje format?	Model może zwrócić inny format, wartości spoza słownika lub komentarz; wynik modelu to dane niezaufane.
Jakie są dwie linie obrony przed prompt injection?	Separacja danych i instrukcja w promptcie systemowym oraz deterministyczne reguły w T-SQL sprawdzające wynik; do tego akceptacja człowieka.

Co przygotować przed zajęciami

- SQL Server 2025 (17.x) z wyjściem HTTPS do `api.anthropic.com` lub `api.openai.com` (sprawdź zaporę i proxy firmowe – to najczęstsza przeszkoda).
- Klucze API dla uczestników: osobne klucze z niskim limitem wydatków ustawionym w konsoli dostawcy; po zajęciach je unieważnij. Koszt całego warsztatu na osobę to zwykle ułamek dolara.
- Aktualne nazwy modeli, wersję API i ceny – popraw je w `dbo.KonfiguracjaLLM` (skrypt 1.1), jeśli się zmieniły.
- Uprawnienia: sysadmin do `sp_configure` (albo włącz opcję wcześniej), `EXECUTE ANY EXTERNAL ENDPOINT`, `CREATE DATABASE`.
- Przejdź cały warsztat raz z każdym dostawcą, którego mają używać uczestnicy.
- Plan B: jedno działające stanowisko prowadzącego na rzutniku, jeśli sieć sali szkoleniowej blokuje ruch wychodzący.

Załącznik. Sprzątanie po warsztacie

Plik / skrypt: cleanup.sql

```
/* Sprzątanie po warsztacie */
USE master;
GO
IF DB_ID(N'AsystentOpinii') IS NOT NULL
BEGIN
    ALTER DATABASE AsystentOpinii SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE AsystentOpinii;
END;
GO
/* Opcjonalnie: wyłączenie wywołań REST na serwerze */
-- EXEC sp_configure 'external rest endpoint enabled', 0;
-- RECONFIGURE WITH OVERRIDE;
```

Po zajęciach unieważnij klucze API w konsoli dostawcy – usunięcie bazy usuwa poświadczenie z serwera, ale nie unieważnia klucza.

Źródła

- Microsoft Learn: sp_invoke_external_rest_endpoint (Transact-SQL) – https://learn.microsoft.com/sql/relational-databases/system-stored-procedures/sp-invoke-external-rest-endpoint-transact-sql
- Microsoft Learn: REGEXP_REPLACE (Transact-SQL) – https://learn.microsoft.com/en-us/sql/t-sql/functions/regexp-replace-transact-sql
- Anthropic: Messages API – https://platform.claude.com/docs/en/api/messages
- Anthropic: API versioning – https://platform.claude.com/docs/en/api/versioning
- Anthropic: Models overview – https://platform.claude.com/docs/en/about-claude/models/overview
- OpenAI: Models – https://developers.openai.com/api/docs/models
- OpenAI: Responses API reference – https://developers.openai.com/api/reference/resources/responses/methods/create
- Estetyka dokumentu zapewniona przez Cloude